Universität Konstanz

Introduction to Computing for the Social Sciences

Session 4 – Software

David Garcia

Winter Semester 2026-2027

Course Outline

- Session 1: Introduction
- Session 2: Information Coding
- Session 3: Data Structures
- Session 4: Software
- Session 5: Algorithms
- Session 6: Recursion
- Session 7: Sorting Algorithms
- Session 8: Time Complexity
- Session 9: Theoretical Computer Science
- Session 10: Machine Learning
- Session 11: Databases
- Session 12: Parallel Programming
- Session 13: Social Science Applications
- Session 14: Exam Review

Outline

1. Programming in a nutshell
2. Software engineering
3. Requirements, specifications, and design
4. Testing

The app that leaked 13,000 photo IDs

Universität Konstanz

MOTIVATING EXAMPLE

Tea, a US app where women warn each other about men they date, asked every new user for a selfie and a photo ID to prove she was a woman. In July 2025 someone posted the address of the storage that held them: **no login, no access rule.** Anyone could download everything.

72,000
images downloaded

13,000
selfies and photo IDs

1 million+
private messages, in a second leak

***"Only moderators may see the ID photos"* was obviously required. Why did nothing enforce it, and which test would have caught it?**

TechCrunch and CNN, 26 July 2025; 404 Media. Many blamed vibe coding; Tea says the code predates 2024.

The 5 millimeters nobody could move

Universität Konstanz

MOTIVATING EXAMPLE

In the 2000s, **Symbian**, the system of Nokia's phones, ran most of the world's smartphones. In 2010 a former Nokia design executive told how his team wanted to move one element of a phone's screen up by 5 mm: *"We can't find the piece of code that could shift it up 5mm."*

20 million
lines of code in one phone

40 million
lines of Symbian released as open source, 2010

11 million
lines of Android at the time

Nobody wrote this code to be unchangeable. What makes working code impossible to change, and what keeps it changeable?

Juhani Risku in The Register, 22 July 2010; All About Symbian, 4 February 2010. In 2011 Nokia replaced Symbian with Windows Phone.

Programming in a nutshell

Universität Konstanz

1. **Programming in a nutshell**
2. Software engineering
3. Requirements, specifications, and design
4. Testing

What is programming?

Universität Konstanz

Definition: Programming

Programming is the process of creating a set of instructions that tell a computer how to perform a task.

Definition: Programming Language

A formal language for writing those instructions, either directly as machine code or as human-readable source code that is translated to machine code automatically.

Vibe coding is programming too

Natural language is turned into machine code automatically: a language model writes the source code, and an interpreter or compiler translates it. The difference is **flakiness**: the same request can give different code, and a different result, each time.

Levels of programming languages

Universität Konstanz

Type	Examples	What it is	The same computation
High-level	Python, R, Java, C	• Written for humans, portable • One line becomes many machine instructions	`area = b * h / 2`

Low-level	Assembly	• Readable names for machine instructions	`LOAD r1,b`
			`LOAD r2,h`
			`MUL r1,r2`
			`DIV r1,#2`

	Machine code	• Binary instructions the CPU executes	`0001001001000101`
			`0010010011101100`
			`10101101001...`

Interpreter vs. compiler

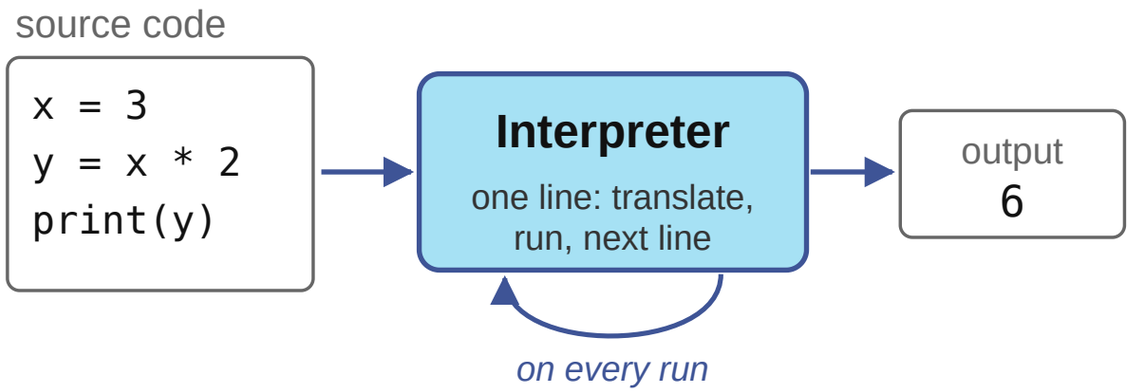
Universität Konstanz

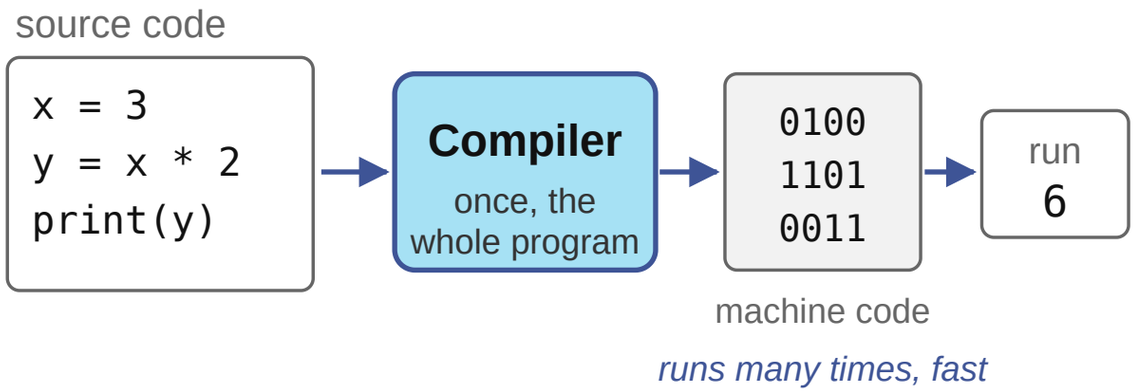
Interpreter: Python, R, JavaScript

- Translates and runs the code **one line at a time**, on every run
- Errors appear when the line runs

Compiler: C, C++, (Java)

- Translates the **whole program** once into machine code
- Many errors appear before it runs





Programming paradigms

Universität Konstanz

Paradigms classify languages and programs by **how** they express a computation

Imperative

Statements that change the program's state, step by step: **how** to compute

Declarative

Describes **what** the result should be, not how to compute it (SQL, session 11)

Object-oriented

Groups data and the instructions that operate on it into **objects**

Functional

The result is the value of functions applied to functions (recursion, session 6)

Python supports all of them: which one you use depends on how you write.

Imperative programming

- The program has a **state**: the current values of its variables
- **Assignments** change the state
- **Control structures** choose what runs next: conditions, loops
- **Functions** name a block of steps, to reuse and check it

```
1  total = 0                  # assignment
2  for age in [34, 41]:       # loop
3      if age >= 18:          # condition
4          total += age
5
6  def mean(xs):              # function
7      return sum(xs) / len(xs)
```

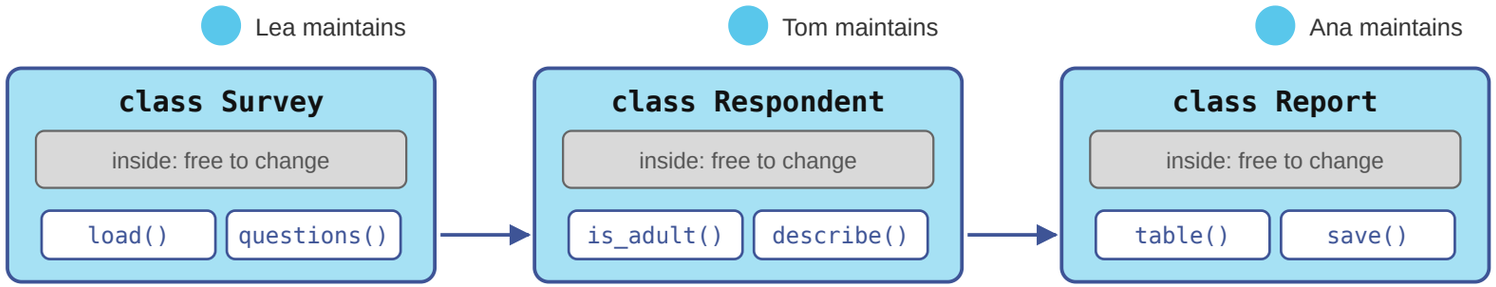
Etymology

imperare, Latin: **to order, to command**. An imperative program is a sequence of orders, carried out one after the other.

Why do we use OOP?

Universität Konstanz

- Large programs are written by **teams** and **maintained** for years
- OOP gives each part of the program **one responsibility**, in one class, owned by one person or team
- Others use a class only through its **methods**: its inside can change without breaking their code, and a bug is fixed in one place



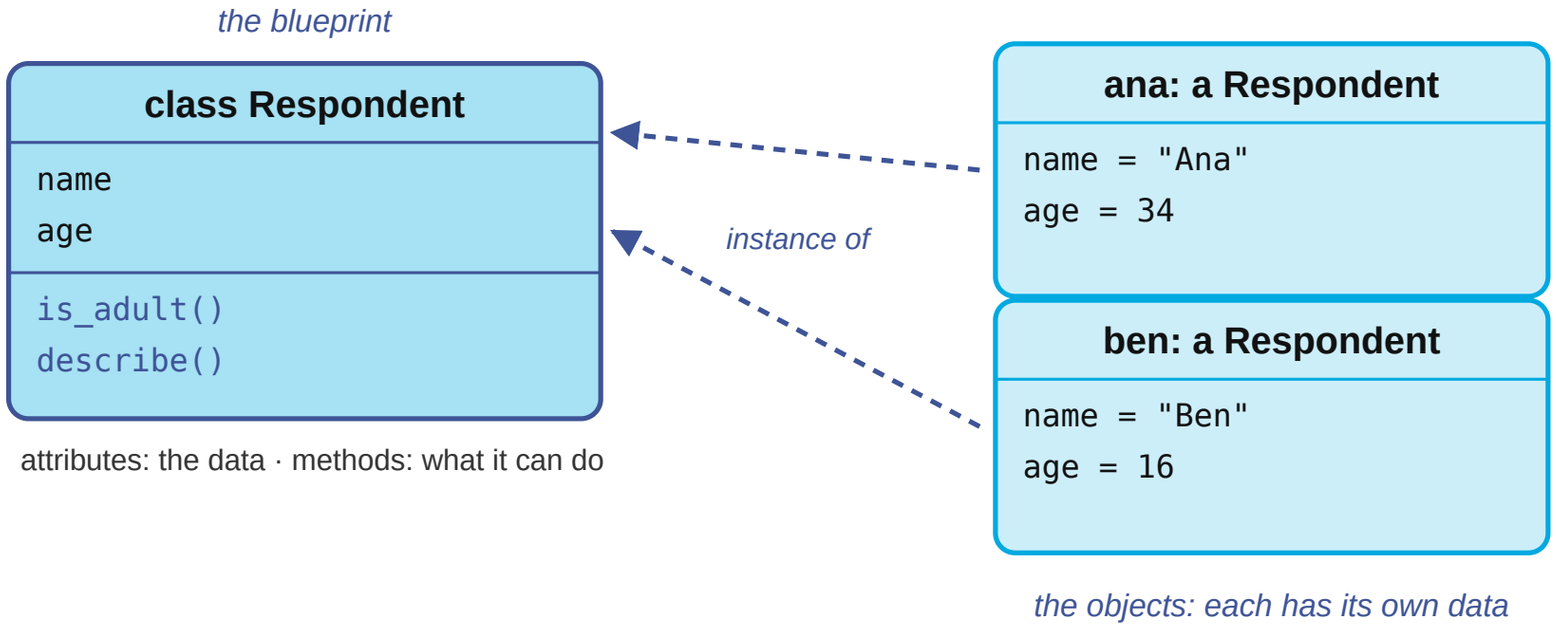
the classes talk only through their methods

A note on scale

For a one-person script, OOP adds little. But it has become a conceptual building block of many programming languages, Python included: every value in Python is an object.

Object-oriented programming

- An **object** bundles **data** (its *attributes*) with the **operations** on that data (its *methods*)
- A **class** is the blueprint; every object is an **instance** of a class
- You query or change an object by calling its methods: `object.method(arguments)`



You already use objects

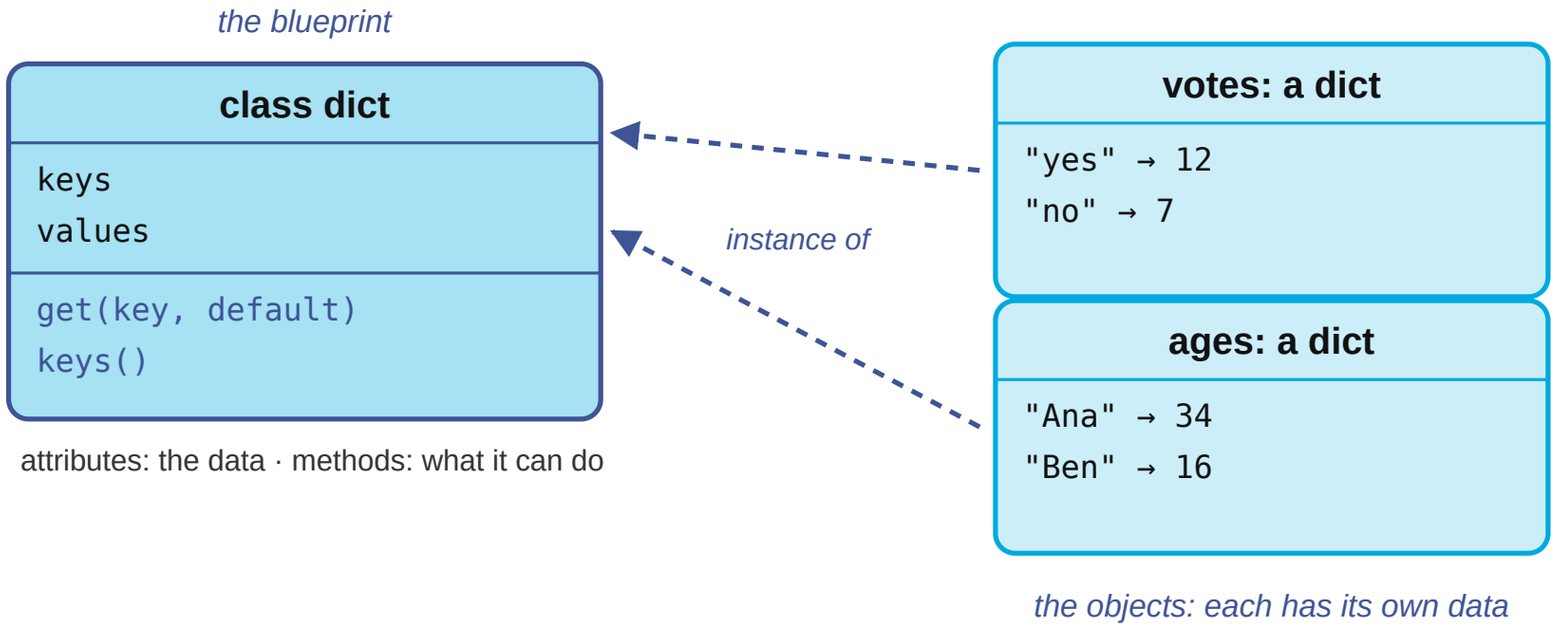
Universität Konstanz

Strings

```
1  s = "Konstanz"
2  s.upper()     # 'KONSTANZ'
3  s.count("n")  # 2
```

Dictionaries

```
1  votes = {"yes": 12, "no": 7}
2  votes.get("no")         # 7
3  votes.get("maybe", 0)   # 0
4  list(votes.keys())      # ['yes', 'no']
```



`s` is a `str` object, `votes` a `dict` object; lists and data frames work the same way.

Your own class and objects

Universität Konstanz

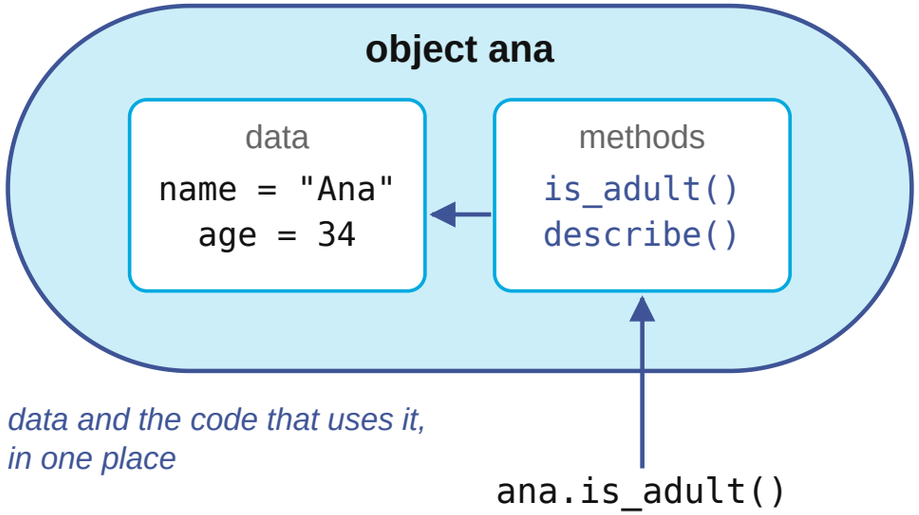
```
 1 class Respondent:
 2     def __init__(self, name, age):       # runs when an object is created
 3         self.name = name                 # attributes
 4         self.age = age
 5
 6     def is_adult(self):                  # methods; self is the object
 7         return self.age >= 18
 8
 9     def describe(self):
10         return f"{self.name}, {self.age} years"
```

```
1 ana = Respondent("Ana", 34)
2 ben = Respondent("Ben", 16)
3 ana.describe()
4 ben.is_adult()
```

```
'Ana, 34 years'
False
```

OOP concepts

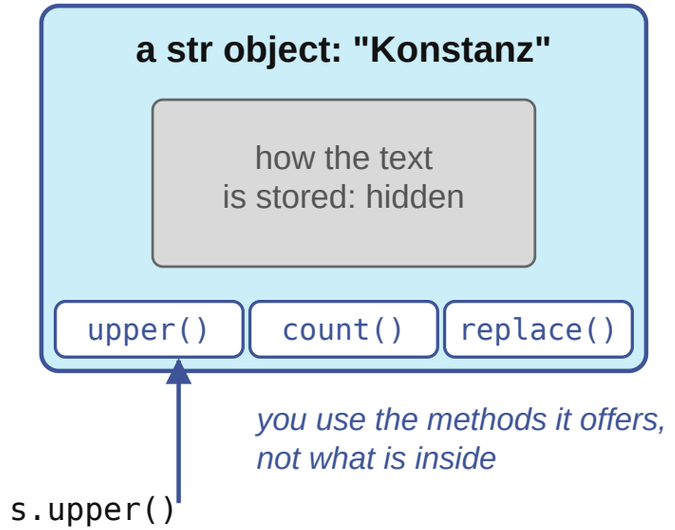
- **Encapsulation:** data and the methods that use it live together in one object




```
1  ana = Respondent("Ana", 34)
2  ana.age           # 34
3  ana.is_adult()    # True
```

OOP concepts

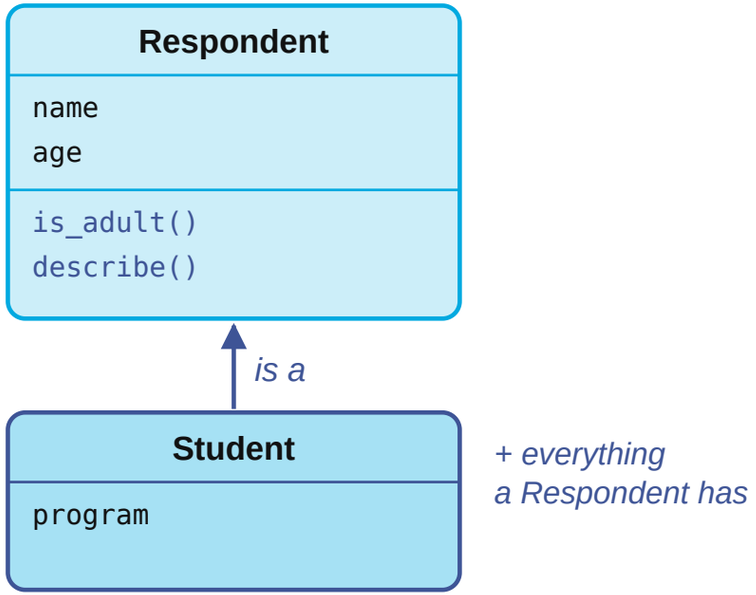
- Encapsulation: data and the methods that use it live together in one object
- **Abstraction:** you use an object through its methods, without knowing how it works inside




```
1  s = "Konstanz"
2  s.upper()   # 'KONSTANZ', but how?
```

OOP concepts

- Encapsulation: data and the methods that use it live together in one object
- Abstraction: you use an object through its methods, without knowing how it works inside
- **Inheritance:** a new class extends an existing one and reuses its methods: a student *is a* respondent

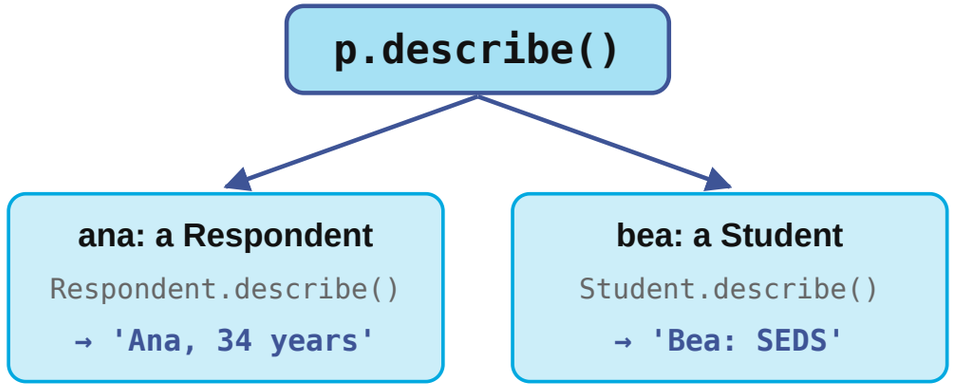



```python
class Student(Respondent):
    def __init__(self, name, age, program):
        super().__init__(name, age)
        self.program = program

bea = Student("Bea", 21, "SEDS")
bea.is_adult()  # True: inherited
```

OOP concepts

- Encapsulation: data and the methods that use it live together in one object
- Abstraction: you use an object through its methods, without knowing how it works inside
- Inheritance: a new class extends an existing one and reuses its methods: a student *is a* respondent
- **Polymorphism:** the same call does the right thing for each class



the same call; each object runs the method of its own class

```
1 class Student(Respondent):
2     def describe(self):
3         return f"{self.name}: {self.program}"
4 ana.describe()  # 'Ana, 34 years'
5 bea.describe()  # 'Bea: SEDS'
```

Software engineering

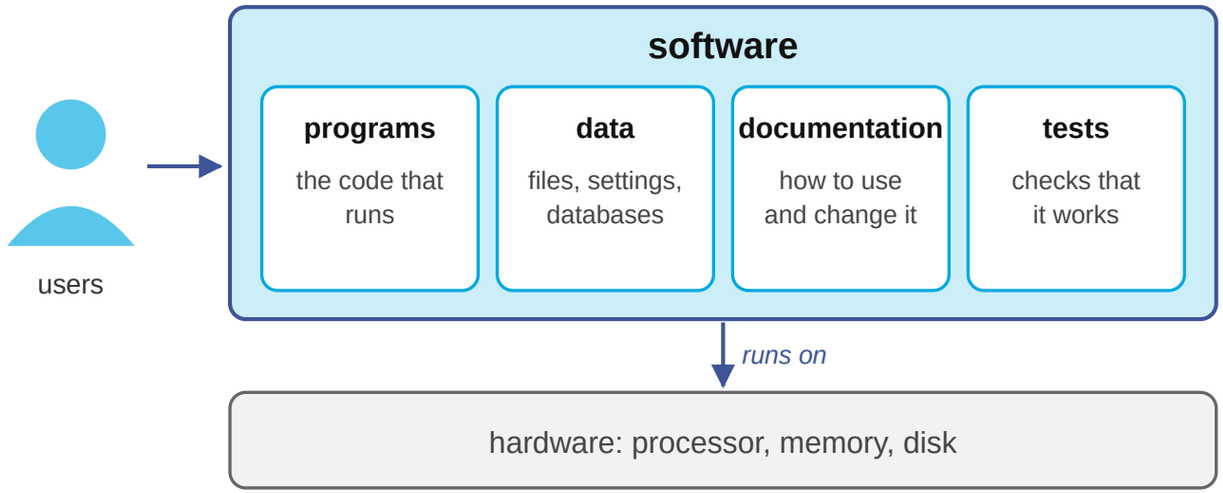
1. Programming in a nutshell
2. **Software engineering**
3. Requirements, specifications, and design
4. Testing

What is software?

Universität Konstanz

Definition: Software

Computer programs, procedures, and possibly associated documentation and data pertaining to the operation of a computer system.



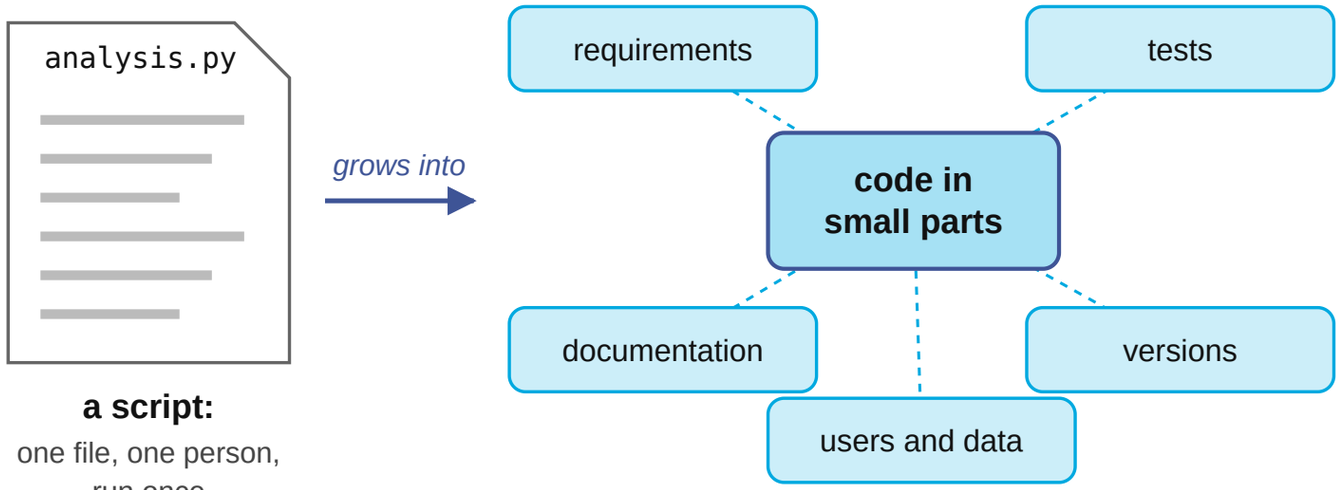
IEEE Std 610.12-1990, *Standard Glossary of Software Engineering Terminology*

From a script to software

Universität Konstanz

Definition: Software engineering

The application of a systematic, disciplined approach to the development, operation and maintenance of software.

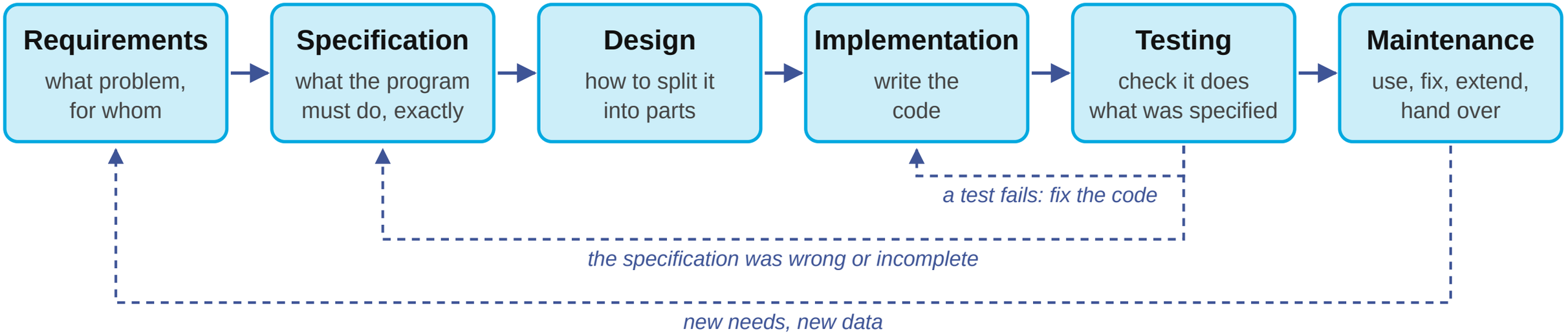


a script:
one file, one person, run once

software: many parts, many people, many years

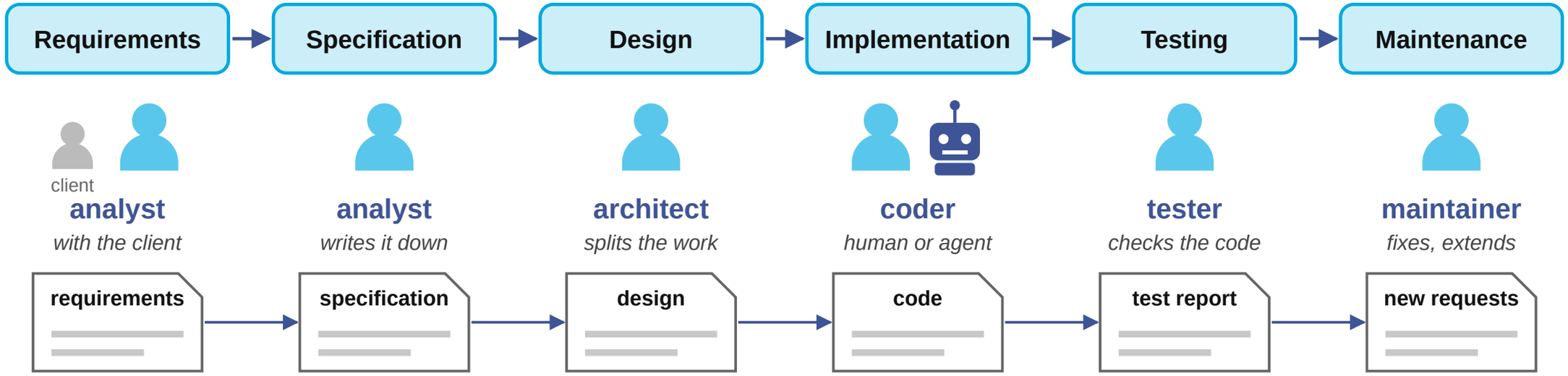
Research code needs this too: it outlives the paper, and others build on it.

A typical software engineering flow



- Real projects go around this loop many times, in small steps (**iterative** development)
- Two steps define correctness: the **specification** says what is right, **testing** checks it

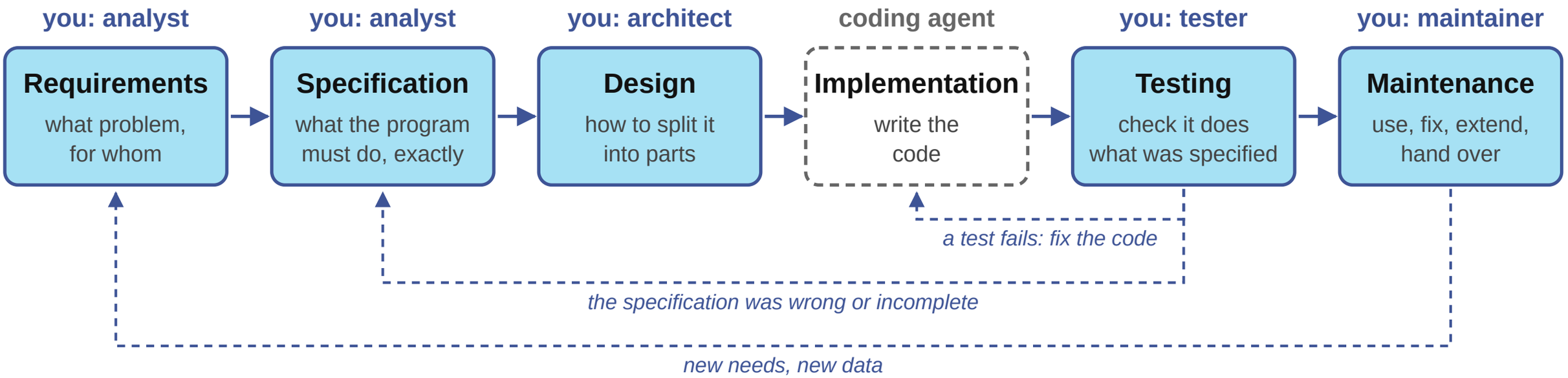
Roles in a software team



each role hands its result to the next one; in a small project, one person plays several roles

- Each stage has an owner: the **analyst** finds out what is needed, the **architect** splits the work, the **coder** implements, the **tester** checks, the **maintainer** keeps it running
- The **coder** can be a person or a coding agent: both work from the same instructions
- In research you often play every role yourself; skipping a role skips its stage

Vibe coding in the flow



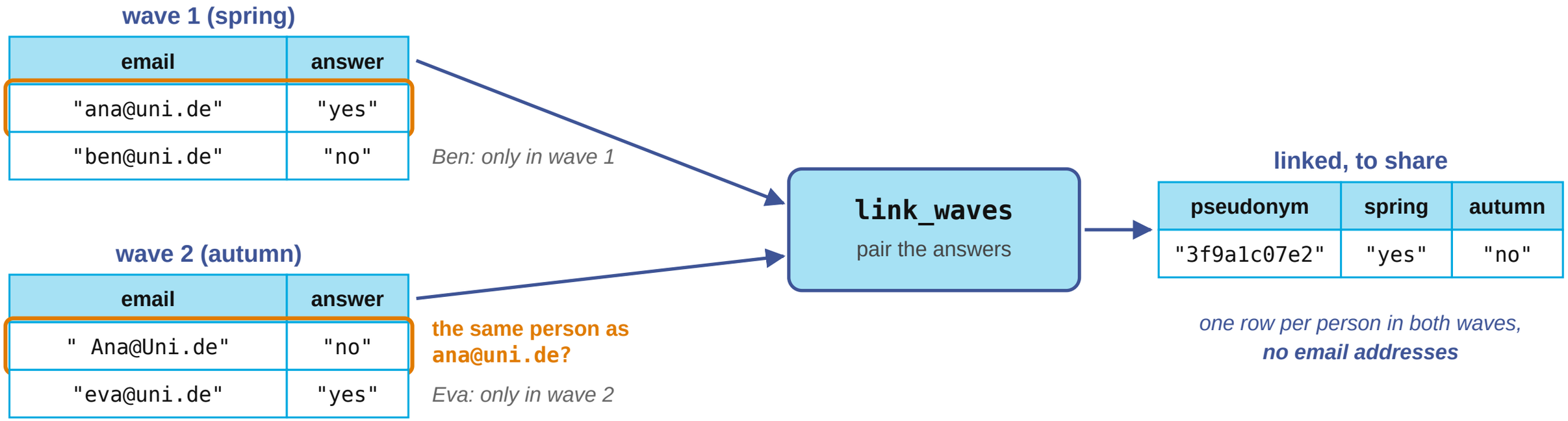
- **Vibe coding**: describe what you want, accept the generated code without reading it, and see whether it seems to work
- A coding agent can take the **coder's** role. **Every other role remains yours**: those stages failed at Tea and at Nokia

Requirements, specifications, and design

1. Programming in a nutshell
2. Software engineering
3. **Requirements, specifications, and design**
4. Testing

Our example: a panel survey in two waves

“We asked the same people the same question in spring and in autumn. I need everyone’s two answers side by side, to see who changed their mind. Please, no email addresses in what I share with the team.” (Ana, the client)



This request goes through every stage and every role with us, from requirements to maintenance

What are requirements?

Universität Konstanz

Definition: Requirements

What the software must achieve, for whom, and under which constraints, in the words of the people who need it.

- The **analyst** collects them from the **client**, who may be a research team, a colleague, or yourself in three months
- They describe the need, not the program. *“Everyone’s two answers side by side, with no email addresses”* is a requirement
- Four questions find most of them. **Who** uses it? With **which data**? What must **never** happen? When is it **done**?
- A misunderstanding found here costs a conversation. Found after the code is written, it costs the code

Functional and non-functional requirements

Universität Konstanz

Functional

What output for what input

- people in both waves are paired, also if they typed their email differently
- an empty wave raises an error

Non-functional

How the program must behave beyond its outputs

- **security**: users see only their own data
- **privacy**: no email addresses in the output
- **safety**: never delete or change the original data
- performance, reproducibility

Tea met its functional requirements. Nobody had written down the non-functional ones.

What is a specification?

Universität Konstanz

Definition: Specification

A precise description of **what** a program must do, without saying **how**.

- The **inputs**, and what may be assumed about them (the *preconditions*)
- The **outputs**, and what is guaranteed about them (the *postconditions*)
- What happens in **edge cases and errors**
- What the program must **never** do
- The analyst writes it for two readers. The **coder** implements it, and the **tester** checks the code against it

You already know the format. Every procedure in this course starts with `Inputs:` and `Result:`

An ambiguous request

“Write a function that links the two waves of our panel survey, so we can see who changed their answer.”

A careful coder asks these five questions. A hurried coder, human or agent, answers them alone and in silence.

1. When are two records the same person? Is `" Ana@Uni.de"` the same as `"ana@uni.de"`?
2. What if someone answered twice in the same wave?
3. What happens to people who answered only once?
4. May email addresses appear in the result?
5. May the function change the lists it was given?

Writing the specification

```
 1 def link_waves(wave1, wave2):
 2     """Pair the answers of the people in both waves.
 3
 4     wave1, wave2: lists of (email, answer) pairs.
 5     Emails match ignoring case and surrounding spaces.
 6     If someone answered twice in a wave, the last answer counts.
 7     Returns: a dict from pseudonym(email) to (answer1, answer2),
 8       only for the people in both waves.
 9     Raises: ValueError if a wave is empty.
10     Never: puts an email in the result or changes the lists.
11     """
```

- Each line answers one of the five questions, and one more line settles the empty case
- Nothing here says **how** to link the waves

The specification is the coder’s instructions

Universität Konstanz

Vague instructions

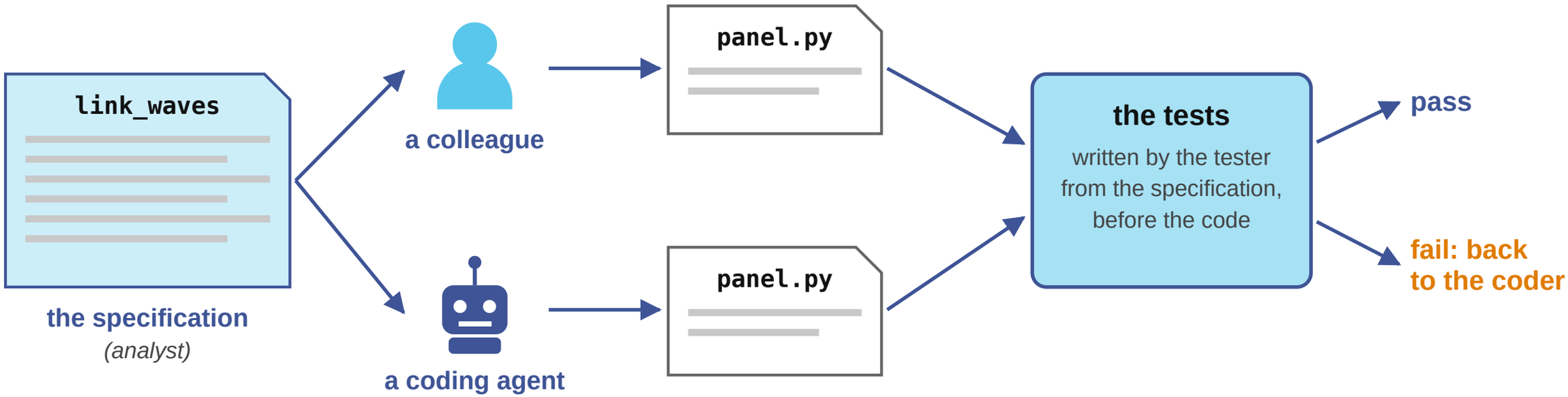
“Write a function that links the two waves.”

Specified instructions

“Write `link_waves(wave1, wave2)`. Each wave is a list of (email, answer) pairs. Match emails ignoring case and spaces. If someone answered twice, keep the last answer. Return a dict from `pseudonym(email)` to the two answers, for the people in both waves. Raise `ValueError` if a wave is empty. Never put an email in the result or change the lists. Use only the standard library.”

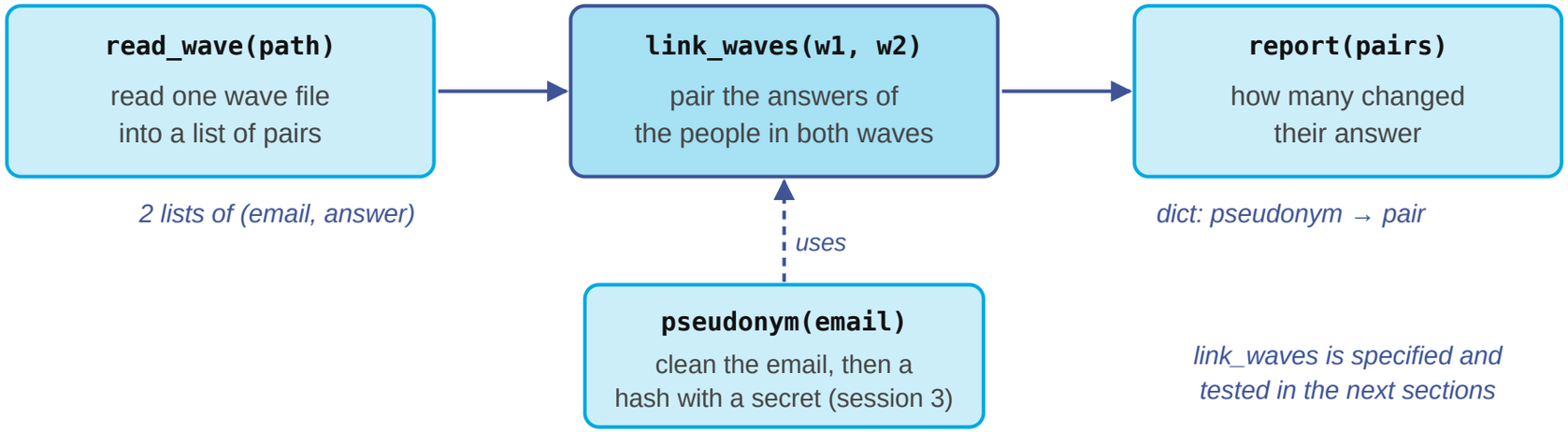
- Precise instructions get better code from any coder, and they give you something to **check** the code against
- A rule that must never be broken needs more than instructions. Enforce it with permissions, backups, and separate test and production data

One specification, any coder



- A colleague and a coding agent receive **the same instructions**, and their code has to pass **the same tests**
- The tester writes the tests from the specification, not from the code, so they check what was asked rather than what was written

Design: small parts with clear connections



- The **architect** splits the program into **small parts**, each with one job and its own specification
- The architect also decides **what data flows between the parts**, and in which data structure
- Small parts are easier to test, easier to replace, and easier to hand to a coder, one specified function at a time

Design: choosing the data structures

Universität Konstanz

The program needs to…	Data structure (session 3)	Example
keep items in order	list	the (email, answer) pairs of a wave
look something up by a key	dictionary	pseudonym → answer, one per wave
check membership fast	set	the pseudonyms in both waves
serve first come, first served	queue	a waiting list for a seminar

A coder, human or agent, writes better code inside a structure the architect chose than inside one it invented on the spot

Testing

1. Programming in a nutshell
2. Software engineering
3. Requirements, specifications, and design
4. **Testing**

What is a test?

Universität Konstanz

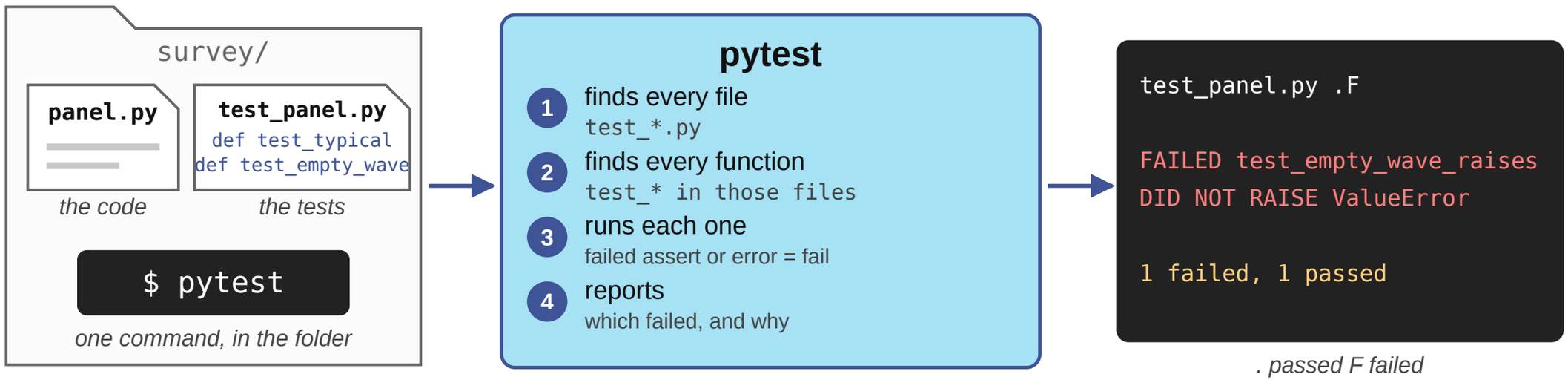
Definition: Test

A small program that runs your code on an input whose correct output you know, and checks that the code produces it.

```
1 w1 = [("ana@uni.de", "yes"), ("ben@uni.de", "no")]
2 w2 = [("ana@uni.de", "no"), ("eva@uni.de", "yes")]
3 assert list(link_waves(w1, w2).values()) == [("yes", "no")]
4 assert "ana@uni.de" not in str(link_waves(w1, w2))
```

- The specification says what should happen. The **tester's** tests check that it does
- A failing `assert` stops the program with an error. That is a loud failure instead of a quiet wrong number

What is pytest?



- **pytest** is a Python package for running tests, installed once with `pip install pytest`
- A test is a function named `test_...` in a file named `test_...`. It **passes** if it runs to the end and **fails** on a failed `assert` or an unexpected error
- One command finds all the tests, runs them, and reports which ones failed and why

Tests with pytest

Universität Konstanz

```python
# test_panel.py
import pytest
from panel import link_waves

W1 = [("ana@uni.de", "yes"),
      ("ben@uni.de", "no")]
W2 = [("ana@uni.de", "no"),
      ("eva@uni.de", "yes")]

def test_typical():
    pairs = link_waves(W1, W2)
    assert list(pairs.values()) == [("yes", "no")]
```

```python
def test_case_and_spaces_are_ignored():
    w2 = [(" Ana@Uni.de", "no")]
    assert len(link_waves(W1, w2)) == 1

def test_no_emails_in_result():
    pairs = link_waves(W1, W2)
    assert "ana@uni.de" not in str(pairs)

def test_empty_wave_raises():
    with pytest.raises(ValueError):
        link_waves(W1, [])
```

Each test checks one line of the specification. The last one uses `pytest.raises` to check that an error is raised

A first implementation meets the tests

```
1  def link_waves(wave1, wave2):
2      first = dict(wave1)
3      return {email: (first[email], answer)
4              for email, answer in wave2
5              if email in first}
```

```
$ pytest -q
.FFF
FAILED test_case_and_spaces_are_ignored
  assert 0 == 1
FAILED test_no_emails_in_result
  'ana@uni.de' is contained here
FAILED test_empty_wave_raises
  DID NOT RAISE ValueError
3 failed, 1 passed
```

A coder's first try. Plausible, short, and wrong three times

- It compares emails exactly, so `" Ana@Uni.de"` becomes a new person and panel members **quietly disappear**
- It keys the result by email, so the privacy requirement is broken without any error
- The fix is to clean each email, key the result by `pseudonym(email)`, and raise an error if a wave is empty. Then all four tests pass

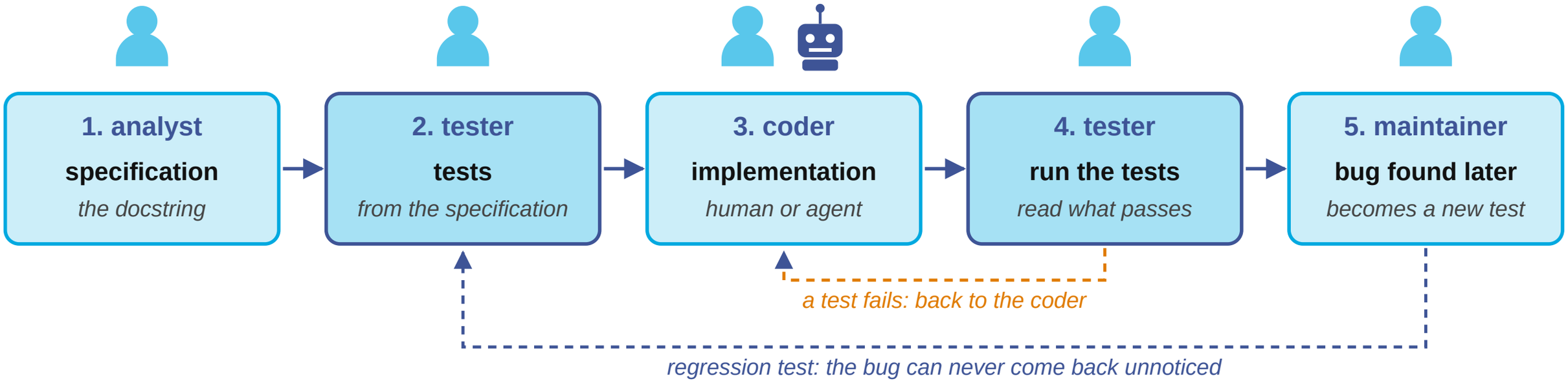
Choosing test cases

Universität Konstanz

Kind	Example input	Expected
Typical	Ana in both waves, Ben and Eva in one	one pair, `("yes", "no")`
Spelling	`" Ana@Uni.de"` and `"ana@uni.de"`	the same person
Duplicates	Ana twice in wave 1	her last answer
Boundary	nobody in both waves	`{}`
Empty	`[]` as a wave	`ValueError`
Must not happen	an email in the result, a changed list	never

Write one test for every line of the specification, and more wherever it draws a line, such as spelling, duplicates, nobody in common, and empty input

Test first, then implement

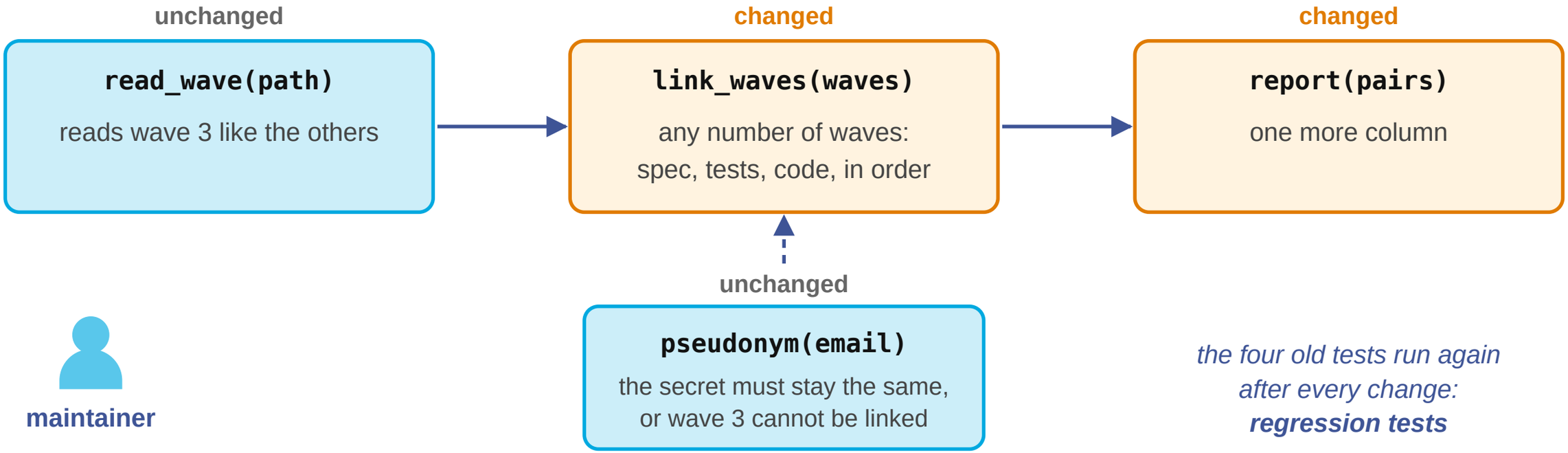


- The tests exist **before** the code, so they record what was meant and not what the code happens to do
- The tests are your measurement. They replace the feeling that the code works with evidence that it does

Maintenance: the third wave arrives

Universität Konstanz

“Great! In spring we run a third wave. Can we link all three?” (Ana, a year later)



The **maintainer** goes through the flow again on a small scale. First the specification changes, then the tests, then the code

What tests cannot do

Universität Konstanz

> *“Program testing can be used to show the presence of bugs, but never to show their absence.”*
> Edsger W. Dijkstra, *Notes on Structured Programming* (1970)

- Tests check only the cases you thought of. A missing requirement produces missing tests
- **Software verification** goes further. It *proves*, with mathematics, that a program meets its specification for **every** input. It is expensive, so it is used where failure is unacceptable, as in aircraft control, railway signaling and operating system kernels
- For research code, the practical combination is a specification, tests, and **reading** the code you did not write

A proof instead of a test: a loop invariant

Universität Konstanz

```
 1  def mean(xs):
 2      """xs: a non-empty list of numbers.
 3      Returns: their mean."""
 4      total = 0
 5      i = 0
 6      while i < len(xs):
 7          # invariant: total == xs[0] + ... + xs[i-1]
 8          total = total + xs[i]
 9          i = i + 1
10      return total / len(xs)
```

Loop invariant

A statement that is true before every turn of a loop

1. **Before the loop**: `i` is 0 and `total` is 0, the sum of no elements. True
2. **Each turn**: `total` gains `xs[i]` and `i` grows by 1, so `total` is again the sum of the first `i` elements. Still true
3. **After the loop**: `i == len(xs)`, so `total` is the sum of all elements and line 10 returns the mean. The list is not empty, so there is no division by zero

A test checks one list. This argument covers **every** non-empty list. Verification tools check such arguments by computer, which is why they need a formal specification

Summary

Universität Konstanz

- A **program** is a precise list of instructions, run by an **interpreter** or translated by a **compiler**
- **Paradigms** differ in how they express a computation. Imperative programs change state step by step, and **objects** bundle data with methods, as in `s.upper()` or your own classes
- **Software engineering** puts coding inside a loop of requirements, specification, design, implementation, testing and maintenance
- **Requirements** say what is needed, the **specification** says exactly what the program must do, and the **design** splits it into small parts
- A **test** runs the code on cases with known answers. Write the tests first, then implement, whoever the coder is
- With coding agents, implementation is cheap. **Everything around it** is the part you cannot delegate

Next session: Algorithms